

Abstract—The digitization of the healthcare sector at lightning speed has created massive amounts of sensitive information about patients, leading to a high need for efficient diagnostics and prediction models. Although ML techniques necessitate the aggregation of all data in one place, resulting in grave privacy concerns and compliance issues, federated learning provides an excellent solution through a decentralized approach in which the data remains on-site within hospitals. However, classical federated learning algorithms such as FedAvg consider each client to be trustworthy and reliable. This is not the case in practice, where the model faces challenges like poisoning and unreliable updates due to differences in computing power. The proposed research work is aimed at the development of a TBFL framework intended to improve the reliability, security, and robustness of the healthcare system. The proposed framework is associated with a multi-step trust evaluation process that considers the computing power of the clients such as the CPU, RAM, bandwidth, and battery health. Additionally, the behavior of the client is considered for the estimation of its trustworthiness. There are four main steps involved in the TBFL process, which include reputation score evaluation, trust score evaluation, threshold-based client selection, and weighted aggregation. From the experimental findings, it is evident that the TBFL approach has an accuracy rate of 97.73%, making it superior to the conventional centralized approaches like Naive bayes, KNN, SVM, Logistic Regression, Decision Tree, Random Forest and ANN models whose accuracy rates stand at 90.29%, 93.95%, 95.74%, 96.42%, 96.53%, 96.85% and 97.53% respectively. The proposed model has been able to filter out substandard contributions and detect any malicious updates, hence offering a balance between high performance and stringent data privacy measures.

I. INTRODUCTION

The rapid growth of digital technology has revolutionized the healthcare industry to create a large number of medical datasets that need to be handled. Contemporary medical systems generate different types of data, including EHRs, images like x-rays, MRI, lab results, genomic information, and real-time patient health data generated by wearables. Such large datasets present great chances for the application of machine learning (ML) models in disease diagnostics and predictions, treatment strategies, and personalized healthcare services. However, centralized storage and processing of such data present critical issues concerning privacy, security, and regulation. Conventional machine learning methodologies utilize centralized systems, wherein information from different healthcare organizations is collected in one place to train the model. Even though this method may provide high accuracy levels due to the availability of massive amounts of information, it entails substantial risks, including data breaches, security issues, and non-compliance with privacy standards such as the Health Insurance Portability and Accountability Act (HIPAA) and General Data Protection Regulation (GDPR). In health care settings, even a small data breach could have catastrophic consequences. Federated Learning (FL) has proven to be an innovative solution for overcoming such problems through decentralized learning techniques. Federated learning allows multiple client devices, for example, hospitals, to jointly train a global model without sharing their respective data. Rather

than sending raw data, clients send model updates (gradients/weights), and a central server combines these model updates to optimize the global model further. This method improves data privacy and minimizes data leakage risks, and thus is ideal for use in the health sector. However, despite the benefits of Federated Learning, there exist numerous obstacles that make it difficult for its implementation in the real world. The first major drawback of this technique lies in the assumption that all clients involved in the training process can be trusted and will give reliable updates. Such an assumption cannot be made because the data distribution among different clients could vary, causing clients to produce low-quality updates. Moreover, some of the clients could have limited computational capabilities or even engage in malicious activities that might cause them to give faulty updates. Another major challenge in federated learning is the presence of non-independent and identically distributed (non-IID) data across clients. In healthcare systems, data collected from different institutions often varies in terms of patient demographics, disease patterns, and clinical practices. This heterogeneity can negatively impact model generalization and slow down convergence. Additionally, system-level heterogeneity, such as differences in computational power, memory, and network bandwidth among clients, further complicates the training process. In order to overcome these difficulties, the current study proposes a trust-based federated learning (TBFL) approach, which improves the performance, robustness, and security of federated learning models in a healthcare environment. The suggested solution employs a trust evaluation algorithm that measures the trustworthiness of clients based on their capabilities, such as computational power, behavior, and performance history. Every client receives a certain trust value that changes dynamically during the training process. Then, a client selection strategy is used based on an established threshold, through which clients with lower trust values are excluded from the training process. A weighted averaging scheme based on trust is also suggested to enhance the efficiency of the process. The key contributions of this work can be summarized as follows. First, a comprehensive trust evaluation mechanism is introduced to assess client reliability in federated learning environments. Second, a threshold-based client selection strategy is proposed to eliminate malicious or low-quality clients from the aggregation process. Third, a trust-based aggregation method is developed to improve model accuracy and robustness. Finally, the proposed framework addresses critical challenges such as non-IID data distribution and system heterogeneity, making it suitable for real-world healthcare applications.

II. RELATED WORK

Federated learning (FL) has been found to be a very good framework for achieving data privacy during machine learning. This is especially true in areas like healthcare, where privacy is an important aspect. Federated learning was initially proposed by H. Brendan McMahan et al. [1] in their paper and was titled Federated Averaging (FedAvg). Federated averaging allows different clients to train one common model using the aggregate of local model updates. Though FedAvg greatly enhances data privacy, it does not consider clients being untrustworthy, which can be a big problem. To ensure privacy in communications, Bonawitz et al. [2] have proposed a

technique called secure aggregation where encryption of updates happens before sending them to the central server. In such a way, it guarantees privacy of individual updates from being accessed by anyone else. However, secure aggregation does not solve the issue of authentication. The main concern with this solution is that it ensures data privacy rather than guaranteeing reliable client updates. The challenge of non-independent and identically distributed (non-IID) data in federated learning has been extensively studied by Yue Zhao et al. [3]. Their work demonstrated that heterogeneous data distributions across clients significantly affect model convergence and accuracy. Although several optimization techniques have been proposed to mitigate this issue, the performance of federated learning systems still degrades in highly skewed data environments, particularly in healthcare applications where patient data varies widely across institutions. Security threats such as Sybil attacks and model poisoning have also been investigated in federated learning. Kairouz Fung et al. [4] analyzed the impact of Sybil attacks, where an adversary creates multiple fake clients to manipulate the aggregation process. Similarly, Eugene Bagdasaryan et al. [5] demonstrated the feasibility of backdoor attacks in federated learning, where malicious clients inject hidden behaviors into the global model without significantly affecting its performance on normal data. These studies highlight the need for robust mechanisms to detect and mitigate adversarial behavior in federated systems. Recent advancements have focused on improving transparency and auditability in federated learning frameworks

III. THE TBFL FRAMEWORK

In this chapter, the proposed TBFL framework is introduced, where a dynamic trust evaluation system is incorporated into the federated learning architecture to improve the reliability and robustness of the model in the healthcare domain. Contrary to the conventional federated learning models that assume the availability of all trusted clients in the system, the presented framework assigns a trust value to every client depending on the client's computational power, behavior, and performance history. The use of such an implementation involves applying the concept of healthcare datasets that will be divided into several clients for testing purposes. In this case, every client is responsible for training their own local models on the basis of the private dataset and sending only the parameters of the model to the server. In turn, the trustworthiness of each client is evaluated by the server based on a trust computation technique.

A) System Architecture

The suggested TBFL approach adopts a decentralized framework that comprises multiple healthcare clients, a central server, and a trust evaluation module. The client-side setup involves multiple healthcare entities including hospitals, clinics, and diagnostic centers. Each client has access to its own database that contains sensitive information such as health records, medical images, and physiological data. Such information is strictly regulated by privacy policies and thus cannot be disclosed directly. The core server functions as a coordination component tasked with the regulation of communication among the clients, distribution of the global model, and

aggregation of the local models. The module for the evaluation of trust is embedded in the server and is essential in determining the reliability of the clients participating in the process. The architecture ensures that raw data remains localized at the client side, thereby preserving privacy. Only model parameters are transmitted between clients and the server. This design not only enhances data security but also reduces communication overhead. The modular nature of the architecture allows easy scalability, enabling the addition of new clients without affecting system performance.

B) Model Overview

The TBFL model is designed to improve the robustness and reliability of federated learning systems by incorporating a trust-based mechanism. In traditional federated learning, all clients are treated equally during aggregation, regardless of the quality of their contributions. This can lead to degraded model performance if some clients provide noisy or malicious updates. This model tries to overcome the above challenge through the introduction of a mechanism that assigns trust scores to clients depending on their behaviors. The trust scores determine which clients should be filtered out as unreliable for contribution while weighting other clients' contributions. This model works in several communication rounds, including local training, trust evaluation, client selection, and aggregation. Trust score is continuously updated based on both present and past behavior of customers. This guarantees that the model responds to any variations in customer performance with time. System heterogeneity is also taken into account in the model by incorporating parameters like computational power and network bandwidth. Overall, the TBFL model enhances model accuracy, improves convergence speed, and provides robustness against malicious clients, making it highly suitable for healthcare applications.

C) Model Design and process flow

The framework for TBFL involves a number of steps, which follow each other sequentially. At the start of the training, a global model is initialized and distributed by the central server among all the clients who participate in the training process. The client machines will provide an update to the server once they are locally trained. The server will calculate the trust score for every client according to some predefined standards. This is followed by a threshold-based selection scheme that selects the reliable clients. The server applies weighted averaging based on trust to update the global model. The new model is subsequently distributed back to the clients for the next iteration of training. This cycle repeats itself until convergence is achieved. The process flow ensures that unreliable clients are filtered out and that reliable clients have a greater influence on the global model. This improves the overall performance and robustness of the system.

D) Mathematical formula

1. *Calculation of reputation score-* The mathematical formulation of the TBFL model includes trust computation, client selection, and aggregation. Trust Score calculation is an important aspect of the TBFL model. All clients are ranked based on certain system metrics such as computing power and communications performance. The following are the main aspects that go into calculating the trust score of each individual client

CPU capability

RAM availability

Battery percentage

Internet speed

Network bandwidth

The score is defined as: $R^i = f(z^{cpu}, r^i, b^i, s^i, bw^i)$

2. *Score normalization-* Since the computed scores may vary significantly across clients, normalization is required to ensure fair comparison. The raw scores are transformed into a standardized range using min-max normalization: $R^{norm} = \frac{(R^i - R_{min})}{R_{max} - R_{min} + \epsilon}$

3. Trust score computation

The trust score of each client plays a crucial role in the proposed Trust-Based Federated Learning (TBFL) framework, as it determines the reliability and contribution potential of individual clients during the aggregation process. In the proposed model, the trust score is computed based on the normalized resource score of each client along with its historical behavior. This ensures that both system capability and consistency of participation are considered while evaluating trust.

The trust score of a client i is defined as: $Trust^i = \frac{R_{good}}{R_{good} + R_{bad}} \times R^{norm}$

Where:-

R_{good} represents the contribution from reliable behavior

R_{bad} represents the contribution from unreliable or malicious behavior.

R^{norm} denotes the normalized score of client i

The value of R_{good} is computed as historical trust of the client. Clients with consistent and honest contributions receive higher reputation values, resulting in a higher initial trust score, while clients with unreliable or malicious behavior are assigned lower values. This approach ensures that both reliability and behavior quality are taken into account rather than just participation or computational power. Overall, the initial trust score provides a fair baseline for further dynamic trust updates during the federated learning process. It is given by: $R_{good} = H$, where H represents

the historical trust value. The historical trust is calculated as the ratio of successful (honest)

contributions to the total number of training rounds: $H = \frac{\sum_{t=1}^T b^t}{T}$

Where:-

T is the total number of training rounds

bi is a binary indicator that takes the value 1 if the client behaves honestly in a given round and 0 if it behaves maliciously.

Similarly, the value of R_{bad} represents the unreliable contribution and is defined by the formula as $R_{bad} = (1 - H)$. The value of R_{bad} represents the negative or unreliable behavior of a client. It captures the proportion of malicious, incorrect, or inconsistent contributions made by the client during training. A higher R_{bad} value indicates poor or suspicious behavior, which reduces the client's trustworthiness in the system. This component is used to penalize clients who do not perform honestly, ensuring that their influence on the global model is minimized. This formulation ensures that clients with inconsistent or malicious behavior receive lower trust scores, even if they have high computational capabilities. In order of this the overall design of the trust computation scheme attempts to strike a balance between efficiency and behavioral stability. If a client always supplies valid updates in each round, it will gain high historical trust, leading to an increase in its total trust level. However, if a client behaves maliciously, it will gradually lose trust.

4) Updated Trust Score

Since client behavior may change over time, it is important to update the trust score dynamically across training rounds. The proposed model incorporates a temporal update mechanism that considers both current and previous trust values. This approach helps in smoothing fluctuations and provides a more stable estimate of client reliability

The updated trust score is computed as: $T_i^{(t+1)} = \frac{(T_i^{(t-1)} + T_i^t)}{2}$, with the initial condition $T_i^0 = 0$

This formulation ensures that sudden changes in client behavior do not drastically affect the trust score. A generalized update function can also be used $T_i^{(t+1)} = f(T_i^{(t-1)}, T_i^t)$, where f represents an averaging or smoothing function. This dynamic updating mechanism enhances the robustness of the trust evaluation process.

5) Client Selection Based on Threshold

In order to ensure that only reliable clients take part in the aggregation process, a threshold-based selection approach is used. Here, the threshold is calculated based on the average trust score of

all the participating clients. The threshold is defined as: $T_{threshold} = \frac{\sum_{i=2}^N T_i}{N}$. Each client is then evaluated against this threshold. If a client's trust score exceeds the threshold, it is selected for participation; otherwise, it is excluded. This decision-making process is represented as: $N^t = \left\{ \left[1, \text{if } T_i^t \geq T_{threshold} \right] [0, \text{otherwise}] \right\}$. The set of selected clients is given by: $N^t = \left\{ i \mid 2 \leq i \leq N, T_i^t \geq T_{threshold} \right\}$. This mechanism effectively filters out low-trust or potentially malicious clients, ensuring that only high-quality contributions are used in the aggregation process.

6) Trust-Based Aggregation

After the selection of the reliable clients, the aggregation of the local updates is done using a trust-weighted approach. In contrast to the federated averaging approach, where the weights for the clients are the same, the weights in the suggested approach are the trust scores. Thus, the clients with a higher trust score will have a greater effect on the global model. The global model

is computed as: $W_{global} = \frac{\sum_{i=1}^C W_i \times T_i}{\sum_{i=1}^C T_i}$, where W_i represents the local model parameters of client i , T_i is the corresponding trust score, and C denotes the set of selected clients. This aggregation strategy improves model accuracy, robustness, and convergence by prioritizing reliable contributions.

E) Workflow of the proposed model

The proposed trust-based federated learning framework operates through a structured four-phase pipeline designed to ensure system integrity and model accuracy. In the initial Reputation Score Evaluation phase, client hardware and network metrics—including RAM, CPU Frequency, and Bandwidth are aggregated to calculate a Normalized Reputation Score. This score is then integrated into the Trust Score Assessment phase, where comprising the Normalised Reputation Score and Historical behaviour tracked via a system log, is combined with the Previous Trust Score to generate an Updated Trust Score. In the third phase, Selection of Efficient Clients, a binary logic is applied where $C_{selected}$ is set to 1 only if the client's trust meets or exceeds a defined Threshold, thereby filtering out unreliable participants. Finally, the Weighted Aggregation phase utilizes the Trusted Clients' Model Updates and their corresponding Selected Clients' Trust values to refine the Global Model, producing a verified Output for the subsequent federated learning iteration.

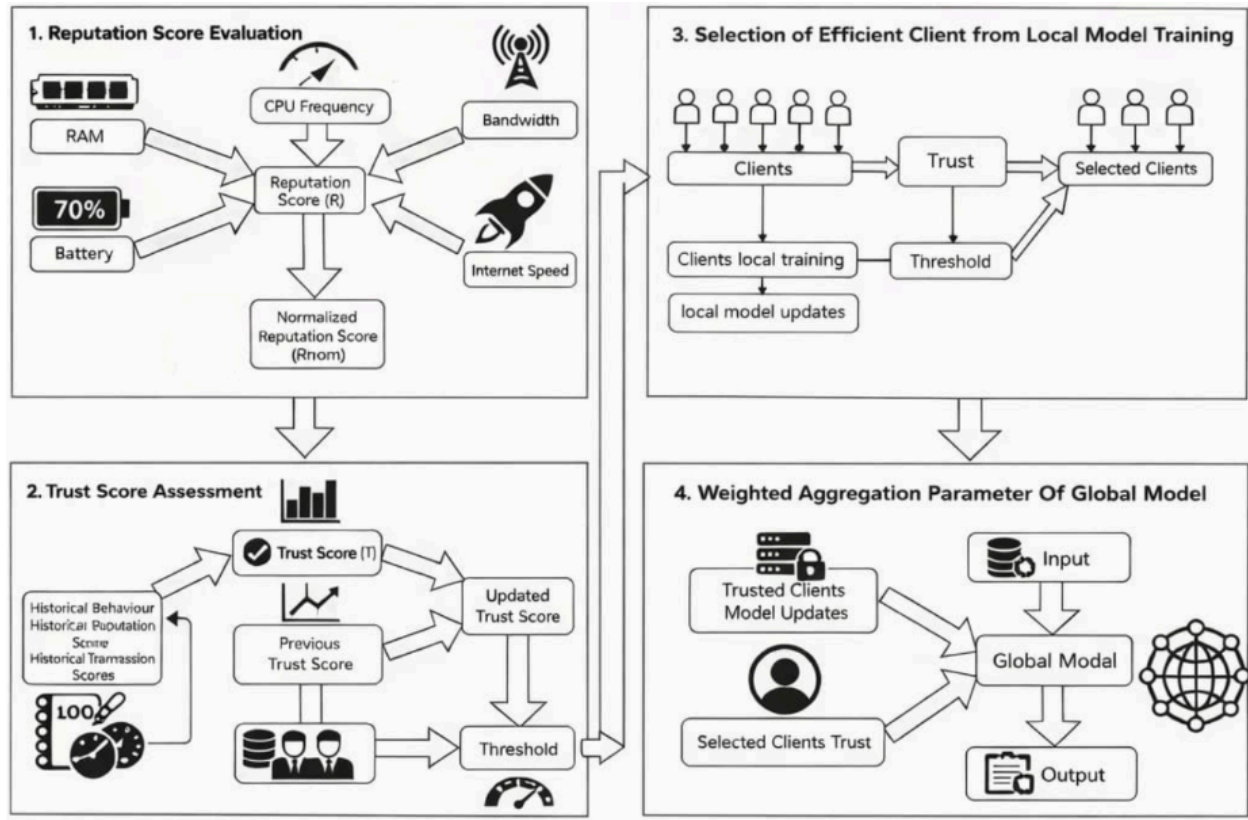


Fig 1.1:- Workflow of the proposed model

F) Flowchart

The complete process flow of the proposed approach can be observed in Fig. 1.2. The approach aims to increase the reliability and robustness of federated learning through the use of a trust-based selection criterion for clients. Initially, the dataset is loaded, and a group of clients is generated in a distributed manner. The resource capability score for each client is then calculated using CPU frequency, ram, internet speed, memory, and bandwidth. These raw scores are then normalized in order to make sure that there is consistency in all cases for all clients. Afterward, there will be a simulation of the behavior of the clients, where both the honest and the malicious clients will be represented. In the event where there are malicious clients, there will be an adversarial update that models the poisoning attack. The trust score of each client is then calculated by combining its normalized resource score and historical reliability. A dynamic threshold is determined using the average trust score, and only those clients whose trust values exceed this threshold are selected for participation in the aggregation process. The selected clients contribute to the global model through a weighted aggregation mechanism, where higher trust scores result in greater influence. Lastly, the aggregate global model will be utilized for making predictions. This guarantees that no negative effect will arise because of any client participating in the learning process who is malicious or unreliable.

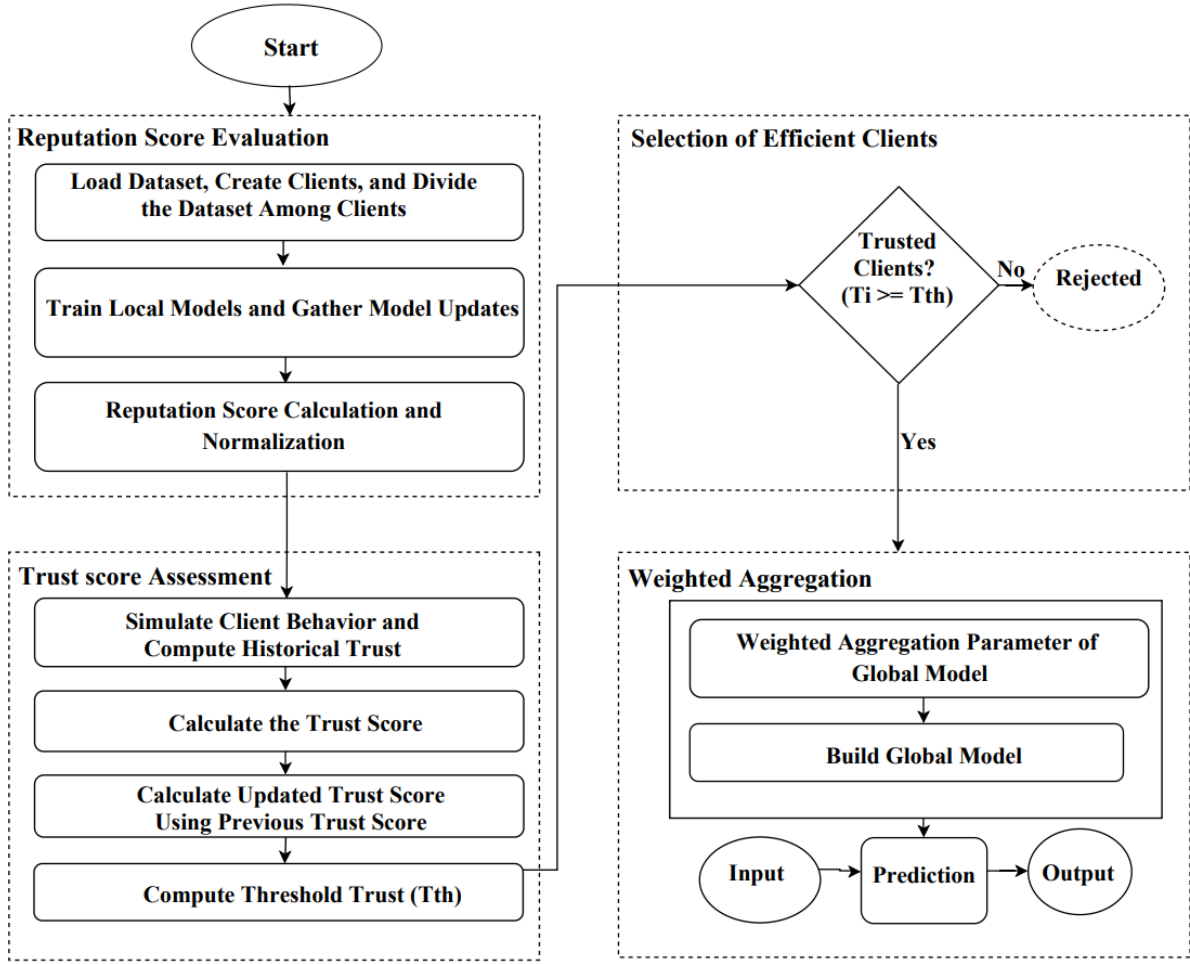


Fig 1.2:- Complete flowchart of the proposed model

G) Algorithm and it's description

A trust-based federated learning technique is provided within the proposed framework in order to guarantee secure and trustworthy collaboration among various healthcare organizations. In this approach, the task of central server involves managing the whole learning process while the various healthcare clients including hospitals and clinics train the model using their own private medical data sets. In place of sharing the data sets among the clients, only the results of training are sent to the central server by the clients.

Algorithm 1. Reputation Score Evaluation:-

The first phase of the proposed framework focuses on evaluating the computational capability of each participating client through a reputation score. This score reflects the efficiency and resource availability of the client system, such as processing power and communication capability. Since clients in a federated environment are heterogeneous, it is essential to normalize these scores to ensure fair comparison. The normalized reputation scores serve as the initial input for further trust evaluation in subsequent phases. The detailed procedure is described below

Input:

- Number of clients N

Output:

- Normalized reputation scores

1. **Initialization step:**
2. $raw_scores \leftarrow []$
3. **for each client** C_i
4. **$i = 1$ to N , do**
5. Compute real score $R^i = f(z^{cpu}, r^i, b^i, s^i, bw^i)$
6. Append R^i to raw_scores
7. **End for**
8. **Compute**
9. $Rmin \leftarrow \min(raw_scores)$
10. $Rmax \leftarrow \max(raw_scores)$
11. **for each client** C_i , **do**
12. $R^{norm} = \frac{(R^i - R_{min})}{(R_{max} - R_{min} + \epsilon)}$
13. **Return** R^{norm}

Algorithm 2. Trust Score Assessment:-

In the algorithm, a trust score is computed for each client to assess its reliability in the collaborative learning process. This phase incorporates both the current reputation score and the historical behavior of the client to provide a dynamic and adaptive trust evaluation mechanism. By considering past interactions, the system is able to identify consistent and reliable clients while reducing the influence of potentially malicious or unstable participants. The trust scores computed in this phase play a crucial role in the selection of clients for global aggregation.

Input:

- Normalized reputation scores (R^{norm})

Output:

- Trust Score ($Trust^i$)

1. **Initialization**
2. $trust_scores \leftarrow \{\}$
3. $trust_history \leftarrow \{\}$
4. **for $i = 1$ to N do**
5. $trust_history[C_i] \leftarrow [0]$
6. Compute Historical factor for each client $H = \frac{\sum_{t=1}^T b^i}{T}$
7. **end for**
8. **for each client** C_i
9. $i = 1$ to N **do**
10. Compute Current trust: $Trust_{curr}^i = \frac{R_{good}}{(R_{good} + R_{bad})} \times R^{norm}$
11. Compute Previous trust $Trust_{prev}^i = trust_history[C_i][i-1]$

12. Compute final trust score: $Trust^i = \frac{Trust_{curr}^i + Trust_{prev}^i}{2}$
13. $trust_history[Ci] \leftarrow Trust^i$
14. Append $Trust^i$ to $trust_history[Ci]$
15. **end for**
16. Return $Trust^i$

Algorithm 3. Selection Of Efficient Clients From Local Training:-

This algorithm is responsible for selecting the most efficient and trustworthy clients based on the computed trust scores. In this algorithm, each client independently trains a local model using its private dataset, and the corresponding updates are sent to the central server. A threshold-based selection mechanism is then applied, where only those clients whose trust scores exceed a predefined threshold are considered for participation in the aggregation process. This ensures that only reliable and high-quality contributions are used in building the global model.

Input:

- Trust Score ($Trust^i$)
- Load dataset (X_i, Y_i)

Output:

- Selected clients $C_{selected}$

1. Initialization
2. $updates \leftarrow []$
3. **for** $i = 1$ **to** N **do**
4. Train local model: $(W_i, b_i) \leftarrow Train(X_i, y_i)$
5. $updates[Ci] \leftarrow (W_i, b_i)$
6. **end for**
7. Client Selection
8. Compute threshold $T_{threshold} = \frac{\sum_{i=2}^N Trust^i}{N}$
9. $C_{selected} = \left\{ i \mid 2 \leq i \leq N, Trust^i \geq T_{threshold} \right\}$
10. Return $C_{selected}, updates(W_i, b_i)$

Algorithm 4. Weighted Aggregation Parameter Of Global Model:-

This algorithm performs the aggregation of local model updates received from the selected clients to construct the global model. Unlike traditional federated learning approaches, the proposed method assigns weights to each client based on their trust scores, thereby giving more importance to reliable clients. This weighted aggregation mechanism improves the overall accuracy and robustness of the global model while minimizing the impact of unreliable updates. The final global model is then used for prediction tasks.

Input:

- Selected clients $C_{selected}$
- $updates(W_i, b_i)$
- Trust score ($Trust^i$)

Output:

• Global model (W, b)

1. Initialization of weight set
2. $Weight_i \leftarrow 0, \forall C_i \in C_{selected}$
3. initialization of aggregation variables
4. $W \leftarrow 0, b \leftarrow 0, D \leftarrow 0$
5. Aggregation Step
6. **for each** $C_i \in C_{selected}$, **do**
7. $Weight_i \leftarrow Trust^i$
8. **end for**
9. Compute
10. $D = \sum_{i=1}^C Weight_i$
11. $(W, b) = \sum_{i=1}^C \frac{\sum weight_i \times (W_i, b_i)}{D}$
12. Prediction
13. $Y = WX + b$
14. Return (W, b)

H) Mathematical Formulation

Let the overall system consist of a dataset $D = (X, y)$ where X represents input features and y represents corresponding labels. The dataset is distributed among N clients denoted as $C = \{C_1, C_2, \dots, C_N\}$ where each client holds a local dataset (X_i, y_i) . Each client independently trains a local model using its private data. The local model parameters of client C_i are represented as (W_i, b_i) while the global model is denoted as (W, b) .

S. No.	Symbol	Description
1	$D = (X, y)$	Complete dataset
2	N	Number of clients
3	C_i	i^{th} client
4	(X_i, y_i)	Local dataset of client C_i
5	(W, b)	Global model parameters (weights and bias)
6	(W_i, b_i)	Local model parameters of client C_i
7	R^i	Raw score of client C_i
8	raw_score	List of all client scores
9	R^{norm}	Normalized score of client C_i
10	R_{min}	Minimum score among all clients
11	R_{max}	Maximum score among all clients

12	R_{good}	Reward component (historical trust)
13	R_{bad}	Penalty component (1 – historical trust)
14	Trust^i	Current trust score of client C_i
15	$\text{Trust}_{\text{curr}}^i$	Current trust update value
16	$\text{Trust}_{\text{prev}}^i$	Previous trust value from history
17	$\text{trust_score}[C_i]$	Final trust score of client C_i
18	$\text{trust_history}[C_i]$	Historical trust values of client C_i
19	Trust^i	Trust value used for selection
20	$T_{\text{threshold}}$	Average trust threshold for client selection
21	C_{selected}	Set of selected clients
22	Weight_i	Aggregation weight of client C_i (equal to trust)
23	D	Normalization denominator ($\sum \text{Weight}_i$)
24	$\sum \text{Weight}_i(W_i, b_i)$	Weighted sum of client updates
25	Y	Final prediction output
26	$Y = W \cdot X + b$	Prediction function

IV. EXPERIMENTAL SETUP

The experimental evaluation of the proposed TBFL architecture is carried out in a Python-based environment using standard machine learning libraries such as NumPy, Pandas, Scikit-learn, Matplotlib, and Seaborn for computation, model training, and visualization. The dataset used in this work is the Parkinson’s UPDRS dataset, which contains biomedical voice measurements for disease prediction. To simulate a federated learning scenario, the dataset is divided into multiple subsets and distributed among five clients, each representing a healthcare entity such as a hospital or diagnostic center. Each client independently trains a local model using a Random Forest classifier on its private data. The distributed nature of the data ensures privacy preservation and reflects real-world healthcare systems. Additionally, simulated resource parameters including CPU capacity, memory, battery level, internet speed, and network bandwidth are assigned to each client to compute a resource-based score, which contributes to evaluating client reliability. Historical behavior is also incorporated to compute trust scores for each client. The training process is executed over multiple communication rounds, where clients perform local training and send their model updates to a central server. The server calculates trust scores, selects clients based on a dynamic threshold, and performs trust-weighted aggregation to generate the final prediction. The presence of unreliable clients is implicitly handled through

lower trust scores rather than explicit noise injection. The performance of the model is evaluated using metrics such as accuracy, training time, prediction time, trust score distribution, and client selection behavior across rounds to validate the effectiveness of the proposed framework.

V. RESULT

The findings from experiments conducted show that the TBFL approach is highly effective in enhancing the efficiency and robustness of federated learning models. Through the trust-based model aggregation technique, clients who have high reliability make more contributions towards the development of the global model, hence improving prediction accuracy. The distribution of trust scores among clients shows a clear distinction between reliable and malicious participants. Clients that consistently provide accurate updates maintain higher trust values, while malicious clients experience a gradual reduction in their trust scores over successive rounds. This dynamic trust adjustment effectively limits the influence of unreliable clients. The convergence of the model is further optimized in the TBFL setting. In contrast to regular federated learning, where the introduction of noise in updates can hamper the convergence process, the trust mechanism discards low-value updates, leading to accelerated and steady convergence. The convergence process is depicted graphically using visual aids like bar graphs and line graphs, which show the fluctuations in the trust value and normalized values among clients. Furthermore, the influence of such malicious clients is minimized to a great extent. Although certain clients may inject noise in their updates, the client selection based on trust ensures that such clients cannot exert dominance in the aggregate process. Accuracy was used as the primary evaluation metric to compare the performance of different machine learning models under both centralized learning and the proposed federated learning framework. The results show that, among the centralized approaches, the Decision Tree model achieved the highest accuracy of 97.5%, followed closely by Artificial Neural Network (97.3%) and Random Forest (96.8%). Logistic Regression also performed strongly with an accuracy of 96.4%, while Support Vector Machine and K-Nearest Neighbors achieved 95.7% and 93.9%, respectively. Naive Bayes showed comparatively lower performance with an accuracy of 90.2%. In contrast, the proposed Trust-Based Federated Learning model achieved an accuracy of 97.7%, making it the best-performing approach overall. This demonstrates that the federated learning framework not only matches but slightly surpasses the best centralized models in terms of predictive performance. More importantly, it maintains a privacy-preserving mechanism by enabling model training across multiple distributed clients without sharing raw data. Thus, the proposed approach effectively balances high accuracy with strong data security, making it highly suitable for sensitive applications such as healthcare and biomedical data analysis.

Table 5.1: Comparison Of Machine Learning Models And Federated Learning Model

Model	Accuracy (%)	Approach Type
Logistic Regression	96.4	Centralized
Support Vector Machine	95.7	Centralized
Decision Tree	96.5	Centralized

Random Forest	96.8	Centralized
Artificial Neural Network	97.5	Centralized
K-Nearest Neighbors	93.9	Centralized
Naive Bayes	90.2	Centralized
Trust Based Federated Learning	97.7	Distributed

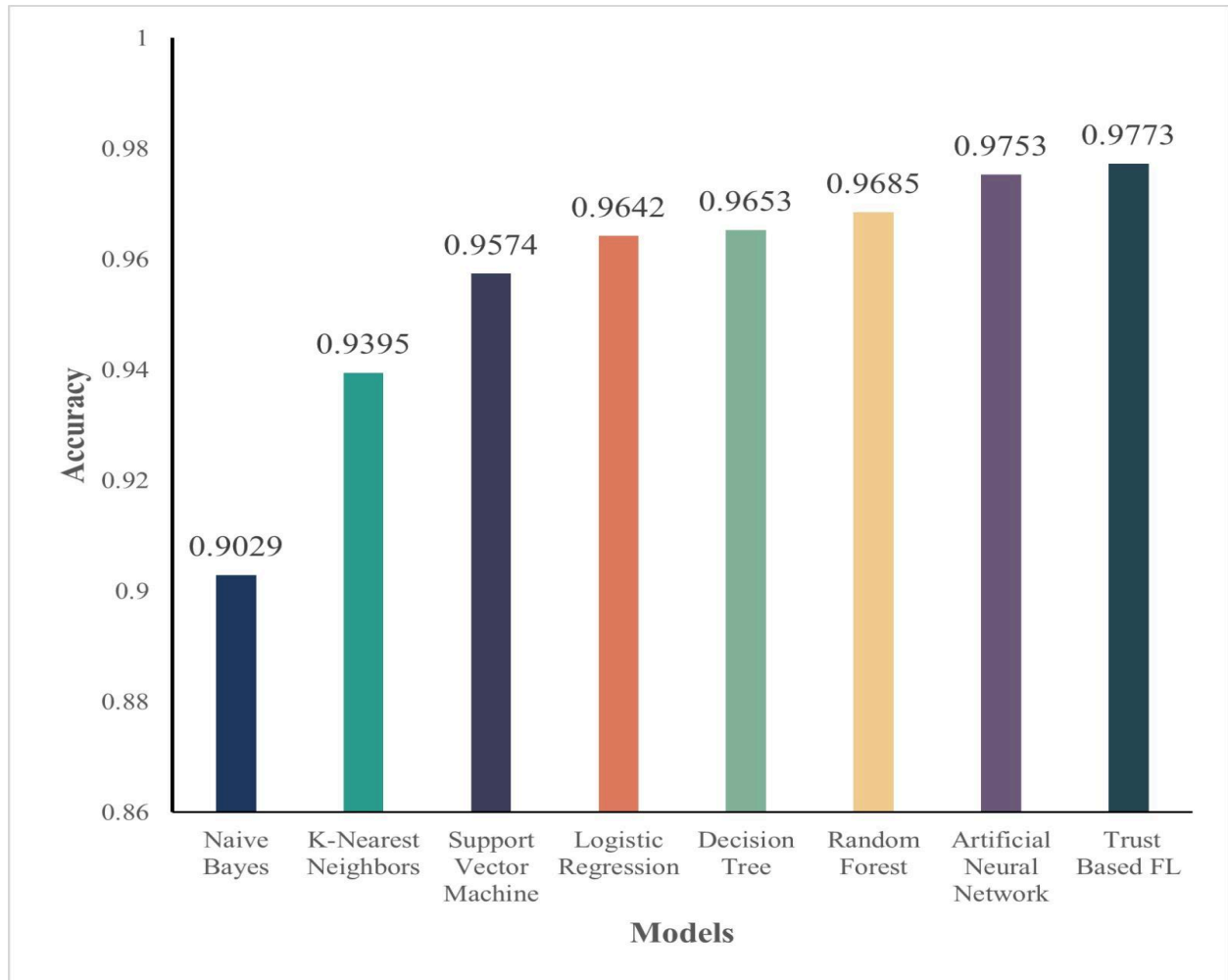


Fig 1.3: Model Accuracy Comparison

Training Latency - The training time results show that Logistic Regression, Decision Tree, KNN, and Naïve Bayes are the fastest models. Random Forest, SVM, and ANN take more time due to higher complexity. The Trust-Based Federated Learning model remains efficient, requiring less time than SVM and ANN while maintaining good performance

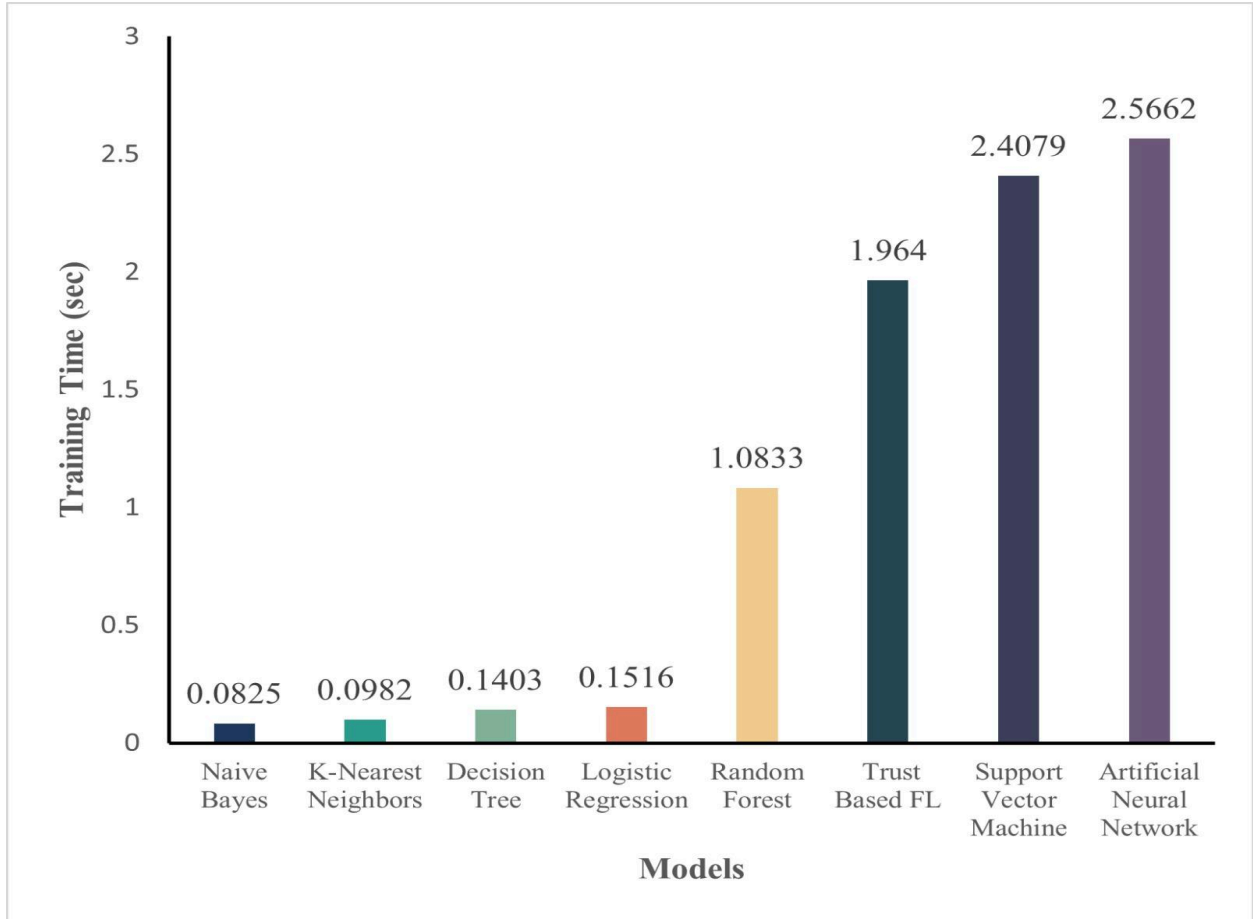


Fig 1.4: Training Latency

Prediction Latency- The prediction time graph of the given models highlights how quickly each model can make predictions after being trained. It is evident that most models, including Logistic Regression, Decision Tree, ANN, and Naïve Bayes have very low prediction times, making them highly efficient for real-time or fast inference tasks. In contrast, KNN shows a significantly higher prediction time because it performs distance calculations with all training data during the prediction phase. Random Forest also requires slightly less time due to the involvement of multiple decision trees in making a final prediction. The Trust-Based Federated Learning model has the highest prediction time among all, as it involves additional steps such as aggregating results from multiple clients and applying trust mechanisms. Overall, the graph demonstrates that while many models are fast in prediction, certain algorithms trade speed for more complex decision-making process

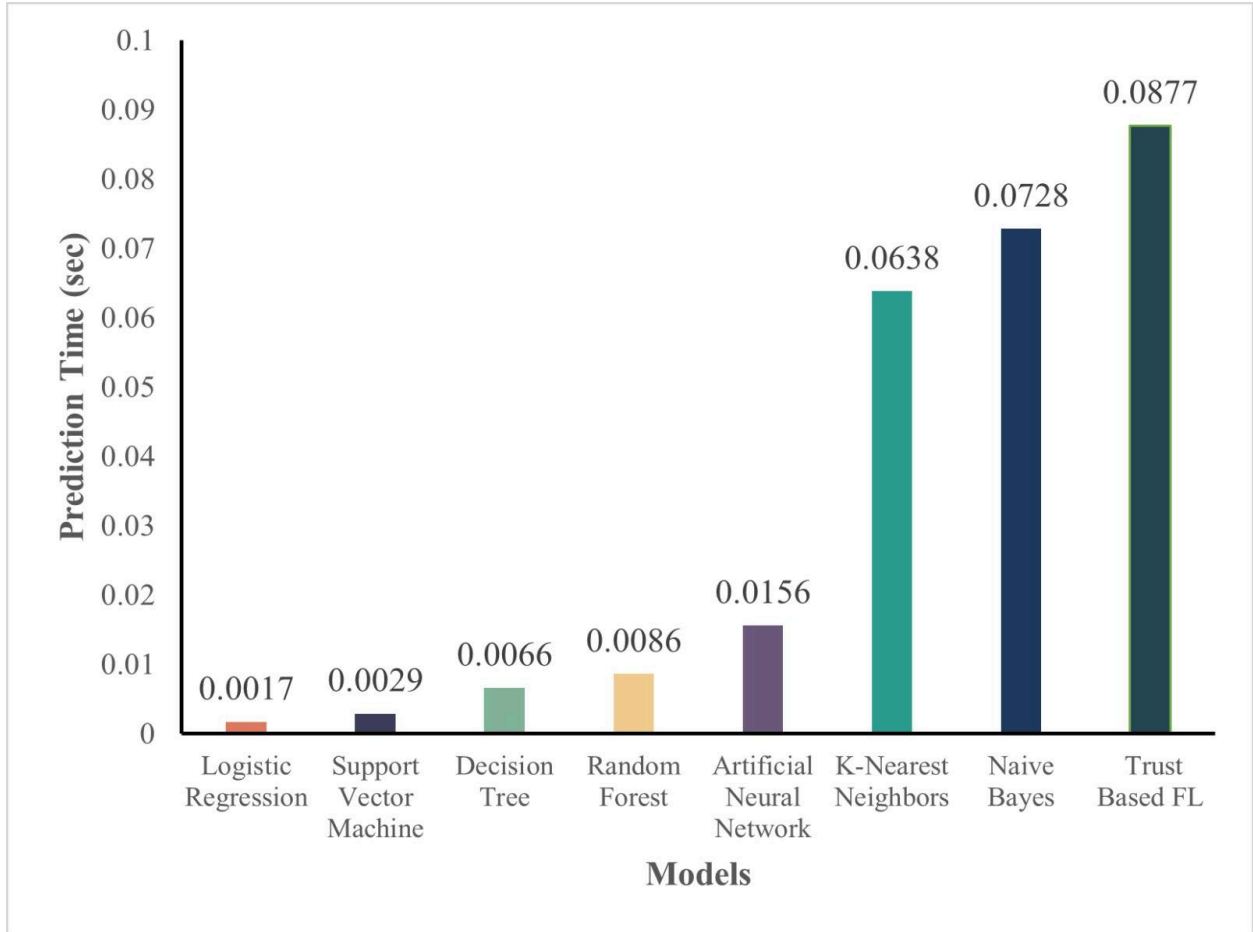


Fig 1.5: Prediction Latency

Cross Validation time- The cross-validation time graph represents the total time taken by each model when evaluated on a new dataset using multiple folds. It shows that simpler models like Logistic Regression, ANN, Decision Tree, and Naïve Bayes require less time due to their fast computation, while models such as SVM, Random forest and KNN take more time because of their complexity and repeated processing in each fold. The Trust-Based Federated Learning model has the highest cross-validation time due to additional steps like aggregation and trust evaluation. Overall, the graph highlights that higher model complexity leads to increased computational time during cross-validation.

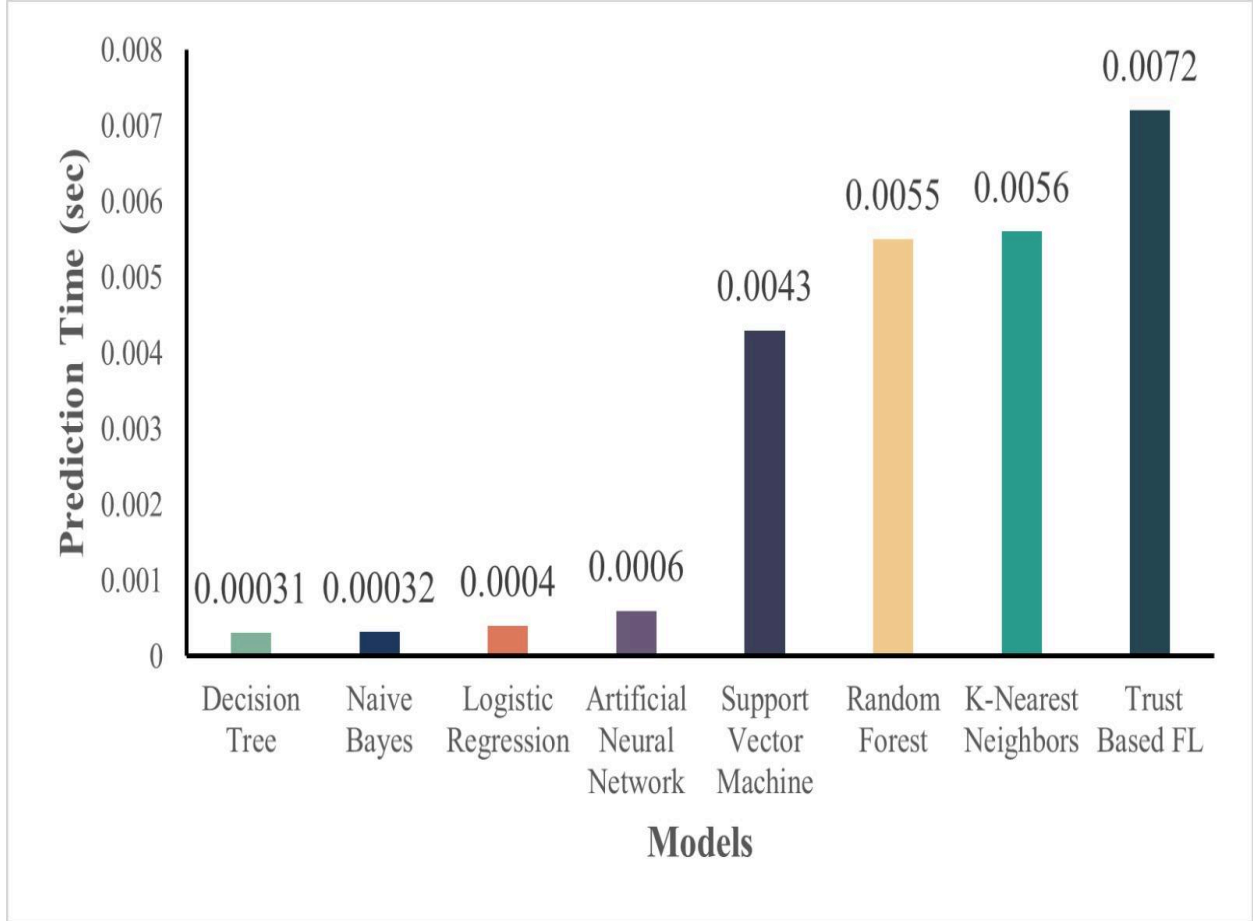


Fig 1.6: Cross Validation Latency

VI. DISCUSSION AND FINDING

Key observations derived from the experimental evaluation of the proposed TBFL framework are summarized as follows. First, the integration of a trust evaluation mechanism significantly enhances the reliability of the system, as it takes into account both the resource capabilities and historical behavior of clients before allowing their participation in global aggregation. Only those clients that achieve higher trust scores contribute to the final model, which improves the overall robustness of the framework. Second, the threshold-based client selection strategy plays a crucial role in eliminating low-performing or less reliable clients. This selective participation not only improves prediction accuracy but also strengthens the system against potentially unreliable updates. The dynamic trust updating mechanism, which combines normalized resource scores and historical trust values, ensures continuous monitoring of client performance across multiple communication rounds, enabling the system to adapt to changing client conditions. Third, the use of trust-weighted aggregation enhances the model's generalization capability, particularly in scenarios where data is distributed and heterogeneous across clients. By assigning higher weights to more trustworthy clients, the framework effectively handles data imbalance and improves the quality of the aggregated model. However, certain limitations are also observed in this approach. The computation of trust scores and management of historical data introduce additional

computational overhead, which may impact scalability in large distributed environments with many clients. Despite this, the overall results demonstrate that the proposed TBFL model provides an efficient and reliable solution for federated learning-based healthcare applications. The framework successfully addresses key challenges such as client heterogeneity, varying resource conditions, and reliability-based client selection, making it a strong candidate for real-world deployment.

VII. CONCLUSION

This project presents a Trust-Based Federated Learning (TBFL) framework designed to improve the reliability and performance of federated learning systems in healthcare scenarios. The proposed model incorporates a trust evaluation mechanism for selecting and aggregating client contributions in a distributed environment. In this framework, each client is evaluated based on multiple resource parameters such as CPU capacity, RAM, battery level, internet speed, and network bandwidth. These factors are combined to compute a resource-based score, which is then normalized and integrated with historical behavior to derive a final trust score for each client. Only clients with sufficient trust levels are allowed to participate in the global model aggregation process. The implementation employs a threshold-based client selection strategy that effectively filters out low-trust or underperforming clients, thereby enhancing the overall quality and robustness of the global model. Additionally, a trust-weighted aggregation mechanism is used, where higher weights are assigned to more reliable clients, ensuring that their contributions have a greater impact on the final prediction. Experimental results demonstrate that the inclusion of trust evaluation improves model accuracy, stability, and generalization performance across multiple communication rounds. This improvement is primarily due to the selective participation of reliable clients and the reduced influence of less dependable ones. Furthermore, the proposed framework maintains data privacy, as raw data remains localized at client devices and only model updates are shared. Overall, the TBFL approach effectively addresses key challenges such as client heterogeneity, varying resource conditions, and reliability in distributed learning environments. Hence, the proposed model serves as a robust and efficient solution for real-world federated learning applications, particularly in the healthcare domain.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2017, pp. 1273-1282.
- [2] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1175-1191.
- [3] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," *arXiv preprint arXiv:1806.00582*, 2018.
- [4] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "The limitations of federated learning in Sybil settings," in *Proc. 23rd Int. Symp. Res. Attacks, Intrusions, Defenses (RAID)*, 2020, pp. 301-316.
- [5] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. 23rd Int. Conf. Artif. Intell. Statist. (AISTATS)*, 2020, pp. 2938-2948.

- [6] A. Yazdinejad, A. Dehghantanha, and G. Srivastava, "A2PFL: Auditable privacy-preserving federated learning framework for electronics in healthcare," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 2527–2535, 2024.
- [7] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, 2020.
- [8] V. Mothukuri, R. M. Parizi, S. Pouriyeh, Y. Huang, A. Dehghantanha, and G. Srivastava, "A survey on security and privacy of federated learning," *Future Gener. Comput. Syst.*, vol. 115, pp. 619–640, 2021.
- [9] R. C. Geyer, T. Klein, and M. Nabi, "Differentially private federated learning: A client level perspective," *arXiv preprint arXiv:1712.07557*, 2017.
- [10] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. 3rd MLSys Conf.*, 2020.
- [11] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol. (TIST)*, vol. 10, no. 2, pp. 1–19, 2019.
- [12] P. Kairouz et al., "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1-2, pp. 1–210, 2021.
- [13] A. Hard et al., "Federated learning for mobile keyboard prediction," *arXiv preprint arXiv:1811.03604*, 2018.
- [14] S. Truex, L. Liu, M. E. Gursoy, and L. Yu, "Demystifying membership inference attacks in machine learning," *IEEE Trans. Services Comput.*, vol. 14, no. 6, pp. 2073–2089, 2021.
- [15] Z. Sun et al., "Can you really backdoor federated learning?," in *Proc. NeurIPS*, 2021.
- [16] R. Shokri and V. Shmatikov, "Privacy-preserving deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2015.
- [17] B. Hitaj, G. Ateniese, and F. Perez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017.
- [18] T. Chen et al., "Federated learning in healthcare: Opportunities and challenges," *arXiv preprint arXiv:2004.11717*, 2020.
- [19] J. Xu et al., "Federated learning for healthcare informatics," *IEEE Intelligent Systems*, vol. 36, no. 4, pp. 56–62, 2021.
- [20] S. Wang et al., "Adaptive federated learning in resource-constrained edge computing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 117–131, 2020.
- [21] R. Hu et al., "Personalized federated learning: A survey," *arXiv preprint arXiv:2103.00710*, 2021.
- [22] J. Lin et al., "Ensemble federated learning for privacy-preserving medical diagnosis," *IEEE Access*, vol. 8, pp. 101552–101561, 2020.
- [23] Y. Zhang et al., "Federated learning for cybersecurity: A survey," *Computers & Security*, vol. 108, 102372, 2022.
- [24] Y. Liu et al., "Federated learning for recommendation systems," *arXiv preprint arXiv:2003.01955*, 2020.
- [25] M. Chen et al., "Federated learning for 5G networks: Opportunities and challenges," *IEEE Network*, vol. 34, no. 5, pp. 24–31, 2020.
- [26] X. Luo et al., "Privacy-preserving federated learning for financial applications," *Future Gener. Comput. Syst.*, vol. 123, pp. 453–463, 2021.
- [27] X. Li et al., "FedBN: Federated learning on non-IID features via batch normalization," *arXiv preprint arXiv:2102.07623*, 2021.
- [28] C. He et al., "FedML: A research library and benchmark for federated learning," *arXiv preprint arXiv:2007.13518*, 2020.
- [29] H. Zhu and Y. Jin, "Multi-objective federated learning," *arXiv preprint arXiv:1908.08049*, 2019.